STRATEGI OPTIMASI KOMPUTASI AWAN

LAPORAN METAHEURISTIK

Perbandingan Kinerja Slime Mould Algorithm (SMA) Dan Aquila Optimizer (AO) pada Penjadwalan Tugas Komputasi Awan dengan Cloudsim Plus



KELOMPOK 2

Disusun oleh:

Ardhi Putra Pradana	5027241022
Aditya Reza Daffansyah	5027241034
Oryza Qiara Ramadhani	5027241084
Ica Zika Hamizah	5027241058
Erlinda Annisa Zahra	5027241108

DEPARTEMEN TEKNOLOGI INFORMASI

INSTITUT TEKNOLOGI SEPULUH NOPEMBER

2026

Abstrak

Penjadwalan tugas pada komputasi awan (cloud computing) bertujuan menentukan mesin virtual yang tepat untuk setiap tugas agar pekerjaan selesai cepat dan biayanya rendah. Laporan ini membandingkan dua algoritma optimasi, yaitu Slime Mould Algorithm (SMA) dan Aquila Optimizer (AO), pada lingkungan simulasi CloudSim Plus 8.0.0 dengan data beban kerja nyata dari Google ClusterData 2019. Pengujian dilakukan pada lima ukuran beban (1.000 sampai 5.000 tugas) dengan tiga kali pengulangan untuk setiap algoritma, sehingga totalnya 30 simulasi.

Hasil menunjukkan bahwa SMA menghasilkan waktu penyelesaian rata-rata yang sedikit lebih cepat (208.211.070 detik dibanding 208.711.485 detik, selisih sekitar 0,24%), sedangkan AO menghasilkan biaya rata-rata yang sedikit lebih rendah (23.909.615 MU dibanding 23.914.429 MU, selisih sekitar 0,02%). Selisih kedua algoritma sangat kecil, sehingga tidak ada algoritma yang unggul mutlak pada seluruh aspek.

Kata kunci: penjadwalan tugas, komputasi awan, Slime Mould Algorithm, Aquila Optimizer, CloudSim Plus, makespan, biaya.

BAB I

Pendahuluan

1.1 Latar Belakang

Penjadwalan tugas pada komputasi awan adalah proses membagikan sejumlah tugas ke sekumpulan mesin virtual (*virtual machine*, VM). Pada infrastruktur yang spesifikasinya beragam (heterogen), masalah ini sulit diselesaikan secara tepat karena jumlah kemungkinan pembagiannya sangat besar. Oleh karena itu digunakan algoritma metaheuristik, yaitu algoritma pencarian yang menghasilkan solusi yang baik dalam waktu yang wajar, meskipun belum tentu yang terbaik secara mutlak.

Dalam praktiknya terdapat dua tujuan yang saling bertentangan:

1. **Waktu penyelesaian (*makespan*)**, yaitu total waktu yang dibutuhkan sampai seluruh tugas selesai. Semakin kecil, semakin baik.

2. **Biaya eksekusi *(cost)***, yaitu biaya sewa mesin virtual selama tugas berjalan. Semakin kecil, semakin baik.

Mesin virtual yang lebih kuat umumnya menyelesaikan tugas lebih cepat, tetapi tarif sewanya lebih mahal. Karena itu diperlukan keseimbangan antara kedua tujuan tersebut.

1.2 Rumusan Masalah

1. Bagaimana kinerja SMA dan AO dalam penjadwalan tugas pada infrastruktur cloud yang heterogen?
2. Algoritma mana yang lebih baik ditinjau dari waktu penyelesaian, biaya, pemanfaatan CPU, throughput, dan keseimbangan beban?

1.3 Tujuan

Penelitian ini bertujuan membandingkan secara empiris kinerja SMA dan AO pada penjadwalan tugas komputasi awan berdasarkan hasil simulasi menggunakan CloudSim Plus dan data Google ClusterData 2019.

1.4 Algoritma yang Dibandingkan

- **Slime Mould Algorithm (SMA**): algoritma yang meniru cara kapang lendir (*Physarum polycephalum*) mencari jalur terbaik menuju sumber makanan.
- **Aquila Optimizer (AO)**: algoritma yang meniru empat pola berburu elang Aquila, mulai dari terbang tinggi menjelajahi area luas hingga menangkap mangsa dari jarak dekat.

1.5 Ruang Lingkup

- Simulator yang digunakan adalah CloudSim Plus 8.0.0.
- Infrastruktur terdiri atas 2 datacenter, 4 host fisik, dan 8 vm.
- Data beban kerja berasal dari Google ClusterData 2019 sebanyak 45.000 tugas.
- Ukuran beban yang diuji adalah 1.000, 2.000, 3.000, 4.000, dan 5.000 tugas.

BAB II
Metodologi

2.1 Data Beban Kerja

Data berasal dari *Google Cluster Workload Trace 2019* (clusterdata_2019_a) dan diambil menggunakan skrip scripts/download_google_trace_sample.py. Agar data bersih, hanya tugas dengan siklus hidup normal yang dipakai, yaitu:

- tugas diajukan (SUBMIT), dijadwalkan tepat satu kali (SCHEDULE), lalu selesai (`FINISH`) tanpa gangguan seperti `EVICT`, `FAIL`, atau `KILL`; dan
- durasi, jumlah core CPU, serta kebutuhan memori bernilai lebih dari nol.

Setiap tugas pada data Google diubah menjadi *cloudlet* (representasi tugas pada CloudSim Plus) dengan aturan berikut.

Atribut pada Data Google	Parameter pada CloudSim Plus	Aturan Konversi
`task_id`	ID Cloudlet	Pengenal unik untuk setiap pekerjaan
`start_time`	Waktu kedatangan (*submission delay*)	Waktu pengajuan relatif terhadap tugas awal, dihitung dalam detik: $(t_submit - t_0) / 10^6$
`execution_duration`	Panjang tugas (MI)	Hasil kali durasi (detik) dengan 1.000 MIPS
`cpu_request`	Jumlah core CPU	Pembagian berbasis kuartil menjadi 1, 2, atau 4 inti CPU

`memory_request_mb`	Kebutuhan memori (MB)	Pembagian berbasis kuartil menjadi 512, 1.024, 2.048, atau 4.096 MB
Atribut pada Data Google	**Parameter pada CloudSim Plus**	**Aturan Konversi**

2.2 Pembagian Data untuk Pengujian

Seluruh 45.000 baris data dipakai secara berurutan tanpa tumpang tindih, sehingga setiap pengujian menggunakan data yang berbeda.

Tahap Uji	Ukuran Beban (Tugas)	Pengulangan	Rentang Baris Data	Total Baris Terpakai
1	1.000	3	1–1.000; 1.001–2.000; 2.001–3.000	3.000
2	2.000	3	3.001–5.000; 5.001–7.000; 7.001–9.000	6.000
3	3.000	3	9.001–12.000; 12.001–15.000; 15.001–18.000	9.000
4	4.000	3	18.001–22.000; 22.001–26.000; 26.001–30.000	12.000

5	5.000	3	30.001–35.000; 35.001–40.000; 40.001–45.000	15.000
Total		**15 per algoritma**	**Baris 1 sampai 45.000**	**45.000**

2.3 Infrastruktur Cloud yang Disimulasikan

Host fisik

Datacenter	**Host**	**Spesifikasi**	**VM yang Ditampung**
Datacenter 1	Host 0	8 core × 3.000 MIPS, RAM 32 GB, bandwidth 10 Gbps	VM 0 dan VM 1
Datacenter 1	Host 1	8 core × 2.500 MIPS, RAM 32 GB, bandwidth 10 Gbps	VM 2 dan VM 3
Datacenter 2	Host 2	12 core × 2.200 MIPS, RAM 48 GB, bandwidth 10 Gbps	VM 4 dan VM 5
Datacenter 2	Host 3	16 core × 1.800 MIPS, RAM 64 GB, bandwidth 10 Gbps	VM 6 dan VM 7

Mesin virtual

VM	Tipe	vCPU	MIPS per Core	Total MIPS	RAM (MB)	Tarif (MU/detik)	Host	Datacenter
VM 0	Small	1	1.000	1.000	2.048	0,010	Host 0	1
VM 1	Small	1	1.000	1.000	2.048	0,010	Host 0	1
VM 2	Medium	2	1.800	3.600	4.096	0,018	Host 1	1
VM 3	Large	4	2.500	10.000	8.192	0,032	Host 1	1
VM 4	Small	1	1.000	1.000	2.048	0,010	Host 2	2
VM 5	Small	1	1.000	1.000	2.048	0,010	Host 2	2
VM 6	Medium	2	1.800	3.600	4.096	0,018	Host 3	2
VM 7	Large	4	2.500	10.000	8.192	0,032	Host 3	2

Penempatan VM ke host bersifat tetap (`StaticVmAllocationPolicy`). Di dalam setiap VM, core CPU dialokasikan khusus untuk satu tugas selama tugas tersebut berjalan (`CloudletSchedulerSpaceShared`).

2.4 Fungsi Penilaian Solusi

Setiap kandidat solusi (pembagian tugas ke VM) dinilai menggunakan satu nilai gabungan (*fitness*) yang memberi bobot 60% pada waktu penyelesaian dan 40% pada biaya. Nilai yang lebih kecil menunjukkan solusi yang lebih baik.

$$\textbf{Fitness} = 0{,}60 \times (\textbf{Makespan} \div \textbf{SkalaWaktu}) + 0{,}40 \times (\textbf{Total Biaya} \div \textbf{SkalaBiaya})$$

Makespan dan biaya dibagi dengan nilai skala agar keduanya berada pada rentang yang sebanding. Nilai skala dihitung sebagai berikut.

- **SkalaWaktu**: jumlah waktu eksekusi tercepat yang mungkin untuk setiap tugas pada VM yang memenuhi syarat.
- **SkalaBiaya**: jumlah biaya setiap tugas bila seluruhnya dijalankan pada VM termurah yang memenuhi syarat.

2.5 Parameter Algoritma

Parameter	Nilai
Ukuran populasi	8
Jumlah Iterasi	12
Bobot makespan : biaya	60 : 40

Mekanisme perbaikan. Jika suatu solusi menempatkan tugas pada VM yang tidak mencukupi (jumlah core atau RAM kurang dari kebutuhan tugas), tugas tersebut dipindahkan secara otomatis ke VM termurah yang masih memenuhi syarat.

2.6 Cara Kerja Singkat Kedua Algoritma

a. SMA

i. Setiap kandidat solusi diberi peringkat berdasarkan nilai *fitness*.
ii. Kandidat berperingkat atas digeser mendekati solusi terbaik yang telah ditemukan, sedangkan kandidat berperingkat bawah tetap mencari secara acak.
iii. Porsi pencarian acak ditingkatkan bertahap seiring bertambahnya iterasi (z = 0,03 + 0,07 × kemajuan iterasi).

b. AO

Pencarian dibagi dua fase dengan total empat strategi.

i. *Fase penjelajahan* (dua pertiga iterasi pertama):
    1. Strategi 1 (terbang tinggi): kandidat dibandingkan dengan posisi rata-rata populasi.
    2. Strategi 2 (terbang mengikuti kontur): kandidat bergerak di antara posisi rata-rata populasi dan solusi terbaik.
ii. *Fase pemanfatan* (sepertiga iterasi terakhir):
    1. Strategi 3 (terbang rendah): pencarian lokal di sekitar solusi terbaik dengan jangkauan yang makin menyempit.
    2. Strategi 4 (menangkap mangsa): penyesuaian kecil (−1, 0, atau +1) di sekitar solusi terbaik.

2.7 Metrik Evaluasi

Metrik	Keterangan
Makespan	Waktu total sampai seluruh tugas selesai (detik); lebih kecil lebih baik
Biaya (*cost*)	Total biaya penggunaan VM (MU); lebih kecil lebih baik
Utilisasi CPU	Persentase pemakaian kapasitas CPU; lebih besar lebih baik

Throughput	Jumlah tugas yang selesai per detik; lebih besar lebih baik
Load imbalance	Tingkat ketidakmerataan beban antar-VM; lebih kecil lebih baik
Deadline miss ratio	Proporsi tugas yang melewati batas waktu; lebih kecil lebih baik

BAB III
Hasil dan Pembahasan

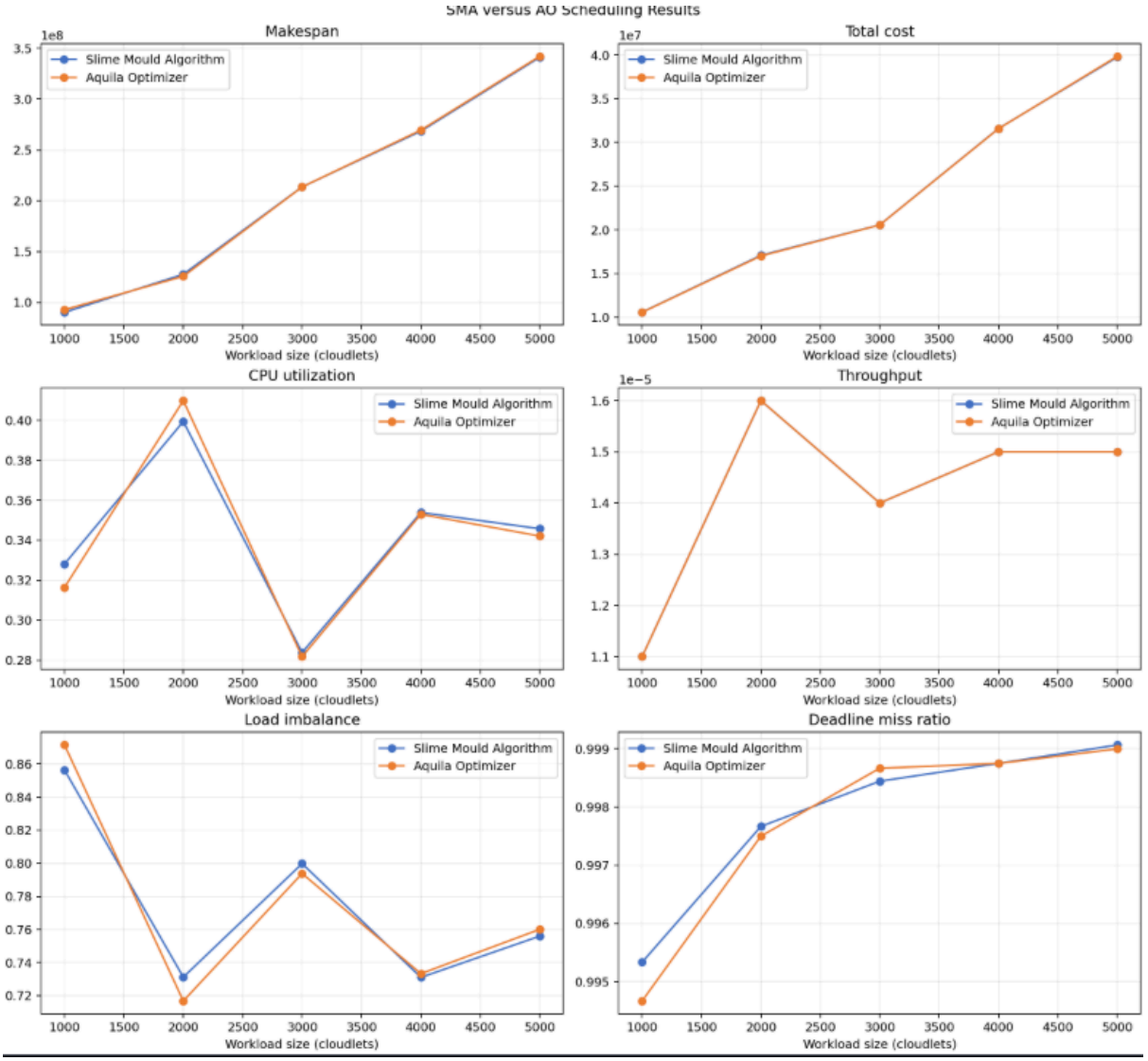
Data hasil simulasi bersumber dari berkas `artifacts/remote-results-5000/summary.csv`. Angka yang dicetak tebal menunjukkan nilai yang lebih baik antara kedua algoritma pada baris yang sama.

3.1 Hasil per Ukuran Beban

Beban (Tugas)	**Algoritma**	**Makespan Rata-rata (detik)**	**SD Makespan**	**Biaya Rata-rata (MU)**	**SD Biaya**	**Utilisasi CPU**	***Throughput* (tugas/detik)**	***Load Imbalance***
1.000	SMA	90.156.631	9.940.662	10.582.237	306.416	0,328	$1{,}11 \times 10^{-5}$	0,856
	AO	92.702.139	9.041.667	10.563.558	322.073	0,316	$1{,}09 \times 10^{-5}$	0,872
2.000	SMA	127.564.850	9.925.407	17.077.487	1.552.298	0,399	$1{,}57 \times 10^{-5}$	0,731
	AO	125.613.682	10.491.309	17.011.049	1.495.579	0,410	$1{,}60 \times 10^{-5}$	0,717
3.000	SMA	213.612.839	39.881.106	20.562.348	3.890.618	0,284	$1{,}46 \times 10^{-5}$	0,800

	AO	213.521.156	38.350.506	20.553.040	3.917.163	0,282	$1,46 \times 10^{-5}$	0,794
4.000	SMA	268.516.123	28.660.976	31.591.108	5.547.716	0,354	$1,51 \times 10^{-5}$	0,731
	AO	269.538.125	29.750.935	31.567.982	5.548.799	0,353	$1,50 \times 10^{-5}$	0,733
5.000	SMA	341.204.909	25.906.196	39.758.966	4.987.278	0,346	$1,48 \times 10^{-5}$	0,756
	AO	342.182.324	22.472.025	39.852.446	5.040.813	0,342	$1,47 \times 10^{-5}$	0,760

SD = standar deviasi dari tiga pengulangan.

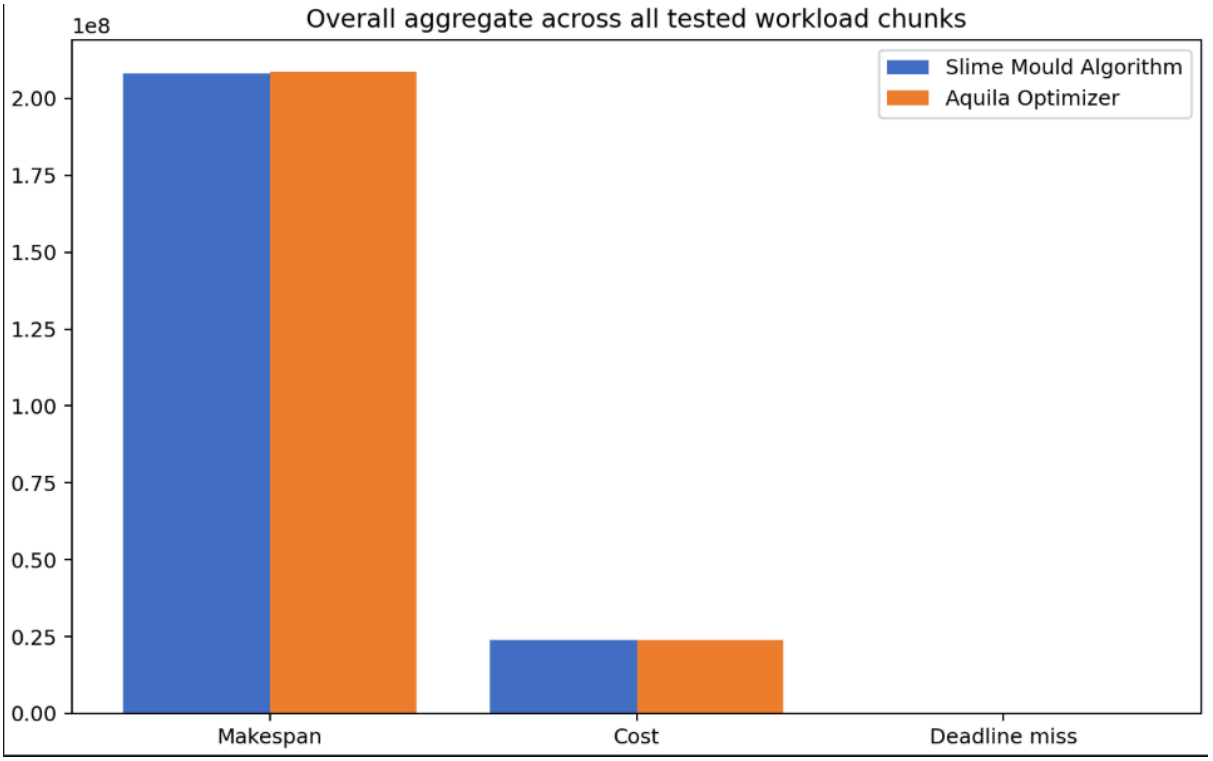


Gambar 1. Perbandingan kinerja SMA dan AO pada setiap ukuran beban (makespan, biaya, utilisasi CPU, throughput, load imbalance, dan deadline miss ratio).

3.2 Hasil Keseluruhan (15 Simulasi per Algoritma)

Metrik	SMA	AO	Lebih Baik
Makespan rata-rata	208.211.070 detik	208.711.485 detik	SMA (lebih cepat ± 0,24%)

SD makespan	97.079.695	97.030.207	AO (sedikit lebih stabil)
Biaya rata-rata	23.914.429 MU	23.909.615 MU	AO (lebih hemat ± 0,02%)
SD biaya	11.287.496	11.329.994	SMA
Utilisasi CPU	0,3422 (34,22%)	0,3406 (34,06%)	SMA
Throughput	$1{,}446 \times 10^{-5}$ tugas/detik	$1{,}444 \times 10^{-5}$ tugas/detik	SMA
Load imbalance	0,7749	0,7751	SMA
Deadline miss ratio	0,9978	0,9977	AO



Gambar 2. Agregat seluruh simulasi. Makespan dan biaya berada pada skala yang berbeda, sedangkan deadline miss ratio bernilai sekitar 1 sehingga batangnya tidak terlihat pada skala ini.

3.3 Pembahasan

a. Waktu penyelesaian (makespan)

Makespan meningkat seiring bertambahnya jumlah tugas, dari sekitar 90–92 juta detik pada 1.000 tugas menjadi sekitar 341–342 juta detik pada 5.000 tugas. SMA menghasilkan makespan yang lebih rendah pada beban 1.000, 4.000, dan 5.000 tugas, sedangkan AO lebih rendah pada beban 2.000 dan 3.000 tugas. Keunggulan SMA diduga berkaitan dengan kecenderungannya mengarahkan tugas ke VM Large (VM 3 dan VM 7) ketika antrian padat. Dugaan ini belum diverifikasi melalui analisis distribusi penugasan.

b. Biaya operasional

AO menghasilkan biaya yang lebih rendah pada beban 1.000 sampai 4.000 tugas, sedangkan SMA lebih rendah pada beban 5.000 tugas. Kemungkinan penyebabnya adalah AO menempatkan sebagian tugas pada VM Small dan Medium yang tarifnya

lebih murah (0,010 dan 0,018 MU/detik). Seperti pada makespan, hal ini masih berupa dugaan.

c. Utilisasi CPU dan *throughput*

Utilisasi CPU kedua algoritma berada pada kisaran 28% hingga 41%, dengan nilai tertinggi pada beban 2.000 tugas (sekitar 40–41%). *Throughput* relatif stabil pada kisaran $1{,}1 \times 10^{-5}$ hingga $1{,}6 \times 10^{-5}$ tugas per detik. Kapasitas yang terbatas, yaitu hanya 8 VM yang harus melayani hingga 5.000 tugas per percobaan, menjadi salah satu faktor yang membatasi nilai-nilai tersebut.

d. Keseimbangan beban

Nilai *load imbalance* berkisar antara 0,72 hingga 0,87. Ketidakseimbangan ini wajar pada infrastruktur heterogen, karena VM Large (4 vCPU, 2.500 MIPS per core) dapat memproses beban yang jauh lebih besar daripada VM Small (1 vCPU, 1.000 MIPS).

e. Rasio keterlambatan (*deadline miss ratio*)

Kedua algoritma memiliki rasio keterlambatan sangat tinggi, yaitu sekitar 0,998, yang berarti hampir seluruh tugas melewati batas waktunya. Kondisi ini menunjukkan bahwa kapasitas 8 VM belum memadai untuk beban yang diuji atau batas waktu yang ditetapkan terlalu ketat, bukan semata-mata kelemahan salah satu algoritma.

f. Catatan tentang signifikansi selisih

Selisih antara SMA dan AO pada seluruh metrik sangat kecil (0,24% untuk makespan dan 0,02% untuk biaya) dan jauh lebih kecil daripada standar deviasi antar-pengulangan. Dengan hanya tiga pengulangan per ukuran beban, perbedaan tersebut belum cukup untuk menyimpulkan bahwa salah satu algoritma secara nyata lebih baik. Hasil ini lebih tepat dibaca sebagai indikasi bahwa kedua algoritma berkinerja sebanding pada konfigurasi yang diuji.

BAB IV
Penutup

4.1 Kesimpulan

1. Seluruh 30 simulasi CloudSim Plus berhasil dijalankan dengan 45.000 baris data unik Google Trace 2019, tanpa pelanggaran batasan kapasitas VM.
2. SMA memperoleh makespan rata-rata yang sedikit lebih rendah, sehingga cenderung cocok bila waktu penyelesaian menjadi prioritas.
3. AO memperoleh biaya rata-rata yang sedikit lebih rendah, sehingga cenderung cocok bila efisiensi biaya menjadi prioritas.
4. Tidak ada algoritma yang unggul pada seluruh metrik, dan selisih kinerja keduanya sangat kecil sehingga dapat dikatakan sebanding.
5. Dengan populasi 8 dan 12 iterasi, kedua algoritma sudah menghasilkan penjadwalan yang stabil dan memenuhi batasan kapasitas VM.

4.2 Saran

1. **Memperbesar parameter pencarian**, misalnya populasi 20 dan iterasi 50, untuk melihat apakah salah satu algoritma menemukan solusi yang lebih baik.
2. **Menguji variasi bobot tujuan** selain 60:40, misalnya 80:20 (fokus waktu) dan 20:80 (fokus biaya).
3. **Menambah jumlah pengulangan** dan melakukan uji statistik agar perbedaan antar-algoritma dapat dinilai lebih meyakinkan.
4. **Meninjau kapasitas infrastruktur atau batas waktu tugas**, mengingat rasio keterlambatan yang sangat tinggi.
5. **Menjalankan simulasi dengan kode algoritma terbaru** dan membandingkannya dengan hasil dasar pada laporan ini.